



UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA E SISTEMISTICA

Primo progetto:

*Sommatore a 16 bit con quattro CLA a 4 bit
collegati in RC*

Studenti:

Emanuele Nisticò

Mat. 251218

Gabriella Xenia Talarico

Mat. 250449

Professoressa del corso:

Stefania Perri

ANNO ACCADEMICO 2025/2026

Contents

1	Introduzione	2
2	Implementazione	3
2.1	Modulo CLA a 4 bit	4
2.2	Sommatore completo	7
3	Schemi logici	8
4	Simulazione logica	10
4.1	Modulo CLA a 4 bit	10
4.2	Sommatore completo	12

1 Introduzione

Il progetto richiede la descrizione in VHDL di un circuito sommatore a 16 bit costituito da quattro moduli Carry Look Ahead (CLA) a 4 bit collegati in cascata (come un Ripple Carry).

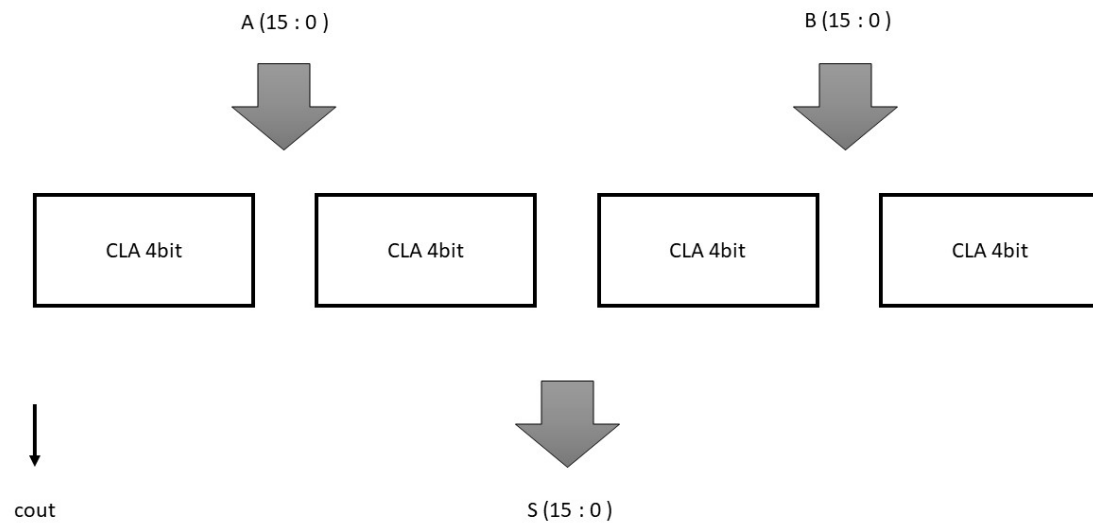


Figure 1: Idea generale del circuito sommatore richiesto

Inoltre è previsto lo svolgimento di una fase di simulazione logica del circuito su almeno sette combinazioni degli input.

2 Implementazione

Dall'idea generale del circuito presentata nell'introduzione si possono intuire alcune caratteristiche implementative sui collegamenti interni ed esterni:

- i bit di A e B devono essere divisi in quattro gruppi da 4 bit e portati in input ai moduli rispettivi; similmente va fatto per i bit di somma S
- gli eventuali bit di riporto in output da un modulo CLA devono essere mandati in input al modulo CLA successivo

Conseguentemente ai ragionamenti appena svolti, ecco come abbiamo interpretato il sommatore richiesto:

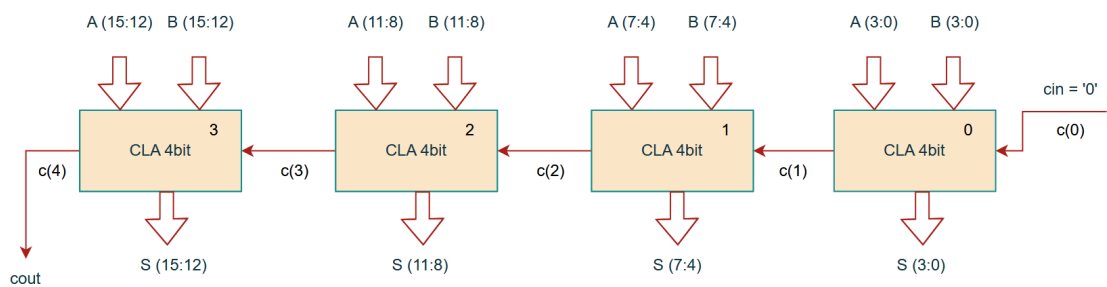


Figure 2: Schema circuitale dettagliato

Già in questa fase abbiamo scelto di utilizzare la notazione tipica dei **bit vector** per la rappresentazione dei riporti. Infatti tale struttura si è resa poi necessaria per una gestione pulita dei segnali nella fase implementativa.

2.1 Modulo CLA a 4 bit

La progettazione del sommatore a 16 bit prevede innanzitutto la descrizione dei moduli CLA a 4 bit di cui è composto.

```
entity CLA4bit is
    Port ( A, B : in bit_vector (3 downto 0);
          cin : in bit;
          cout : out bit;
          S : out bit_vector (3 downto 0));
end CLA4bit;

architecture MyCLA4bit of CLA4bit is
    signal p, g : bit_vector (3 downto 0);
    signal c : bit_vector (4 downto 0);
begin
    p <= A xor B;
    g <= A and B;
    c(0) <= cin;
    c(1) <= g(0) or (p(0) and c(0));
    c(2) <= g(1) or (p(1) and g(0)) or (p(1) and p(0) and c(0));
    c(3) <= g(2) or (p(2) and g(1)) or (p(2) and p(1) and g(0)) or (p(2) and p(1) and
    ↪ p(0) and c(0));
    c(4) <= g(3) or (p(3) and g(2)) or (p(3) and p(2) and g(1)) or (p(3) and p(2) and
    ↪ p(1) and g(0)) or (p(3) and p(2) and p(1) and p(0) and c(0));
    cout <= c(4);
    S <= p xor c(3 downto 0);
end MyCLA4bit;
```

Di seguito l'immagine dello schema logico relativo al circuito appena descritto (per maggiori dettagli si faccia riferimento alla sezione 3):

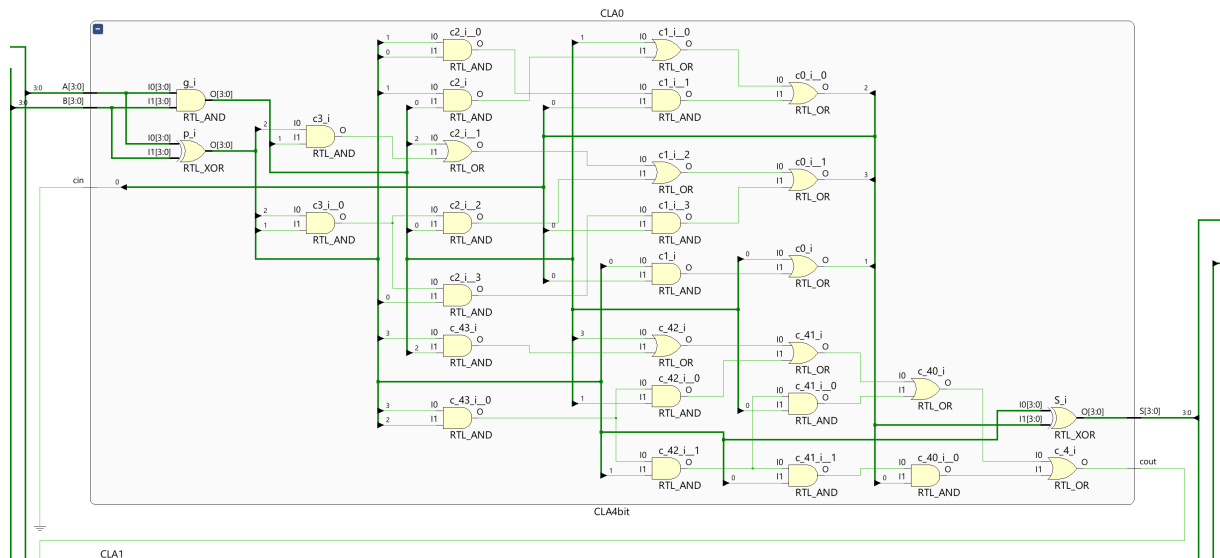


Figure 3: Schema logico del componente CLA

Nel codice sono state prese per assunto le nozioni teoriche riguardanti il modulo CLA, nella seguente sezione si vogliono fare alcuni cenni sul funzionamento di questo sommatore e sui suoi vantaggi e svantaggi:

Sommatore *Carry Look Ahead* (CLA)

Il Carry Look Ahead è un circuito sommatore che si contrappone al Ripple Carry in quanto cerca di eludere il ritardo temporale dovuto alla propagazione dei riporti. Per evitare la propagazione, nel CLA i riporti vengono calcolati parallelamente sfruttando le formule espresse presentate nel seguito.

Consideriamo la generica formula del riporto in un circuito Full Adder:

$$c_{out} = g + p \cdot c_{in} \quad (1)$$

Dove $g = A \oplus B$ e $p = A \cdot B$.

Nel caso di più FA collegati a cascata come nella struttura del sommatore *Ripple Carry* (RC) l'espressione dei riporti $c_1, c_2, c_3, \dots$ sarà quindi:

- $c_1 = g_0 + p_0 \cdot c_0$
- $c_2 = g_1 + p_1 \cdot c_1$
- $c_3 = g_2 + p_2 \cdot c_2$
- $\dots$

Dunque il sommatore RC richiede per lo svolgimento dell' n -esima somma, il calcolo degli $n - 1$ riporti precedenti (il primo $c_0 = c_{in}$ è un dato).

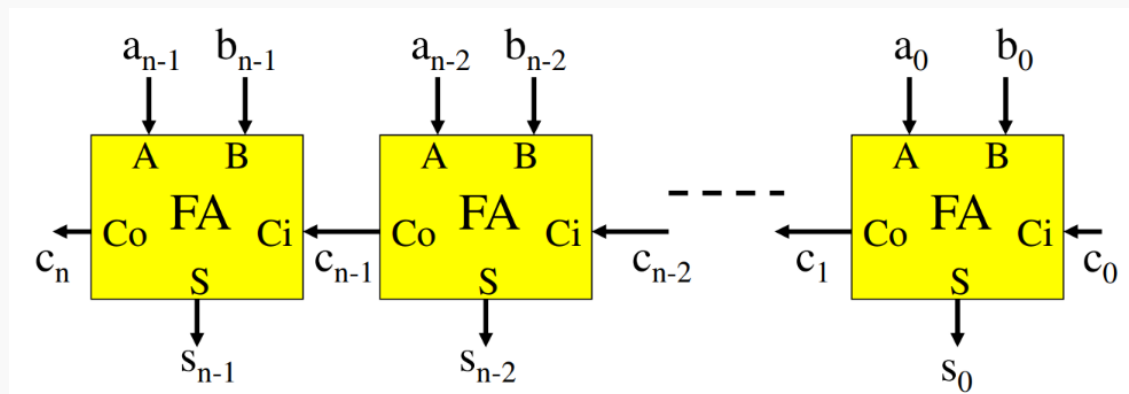


Figure 4: FA collegati a cascata (tipo Ripple Carry - RC), immagine tratta dal *Materiale del corso*

Il sommatore CLA invece calcola in parallelo tutti i riporti sfruttando le formule espresse per il calcolo dei riporti. Queste si ottengono sostituendo nell'espressione dell' i -esimo riporto quella dell' $i-1$ -esimo:

- $c_1 = g_0 + p_0 \cdot c_0$

- $c_2 = g_1 + p_1 \cdot (g_0 + p_0 \cdot c_0) = g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0$
- $c_3 = g_2 + p_2 \cdot (g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0) = g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0$
- ...

Il circuito del CLA è dunque molto più complicato rispetto a quello di un RC e la sua realizzazione è osteggiata da numerosi limiti fisici legati a:

- FAN-IN (numero massimo di ingressi che può gestire una porta logica), infatti il riporto di un certo livello dipende da tutti i segnali propagate e generate dei sottostanti livelli, aumentando così via via la dimensione delle porte.
- FAN-OUT (numero massimo di ingressi di altre porte che può pilotare l'uscita di una porta logica senza degradare i segnali), dato che i vari segnali propagate e generate dovranno andare a pilotare diversi livelli logici successivi

2.2 Sommatore completo

Una volta implementato il generico modulo CLA, siamo passati all'implementazione del sommatore a 16 bit completo. In questa seconda fase abbiamo dunque utilizzato un approccio *gerarchico*. Infatti, seguendo la sintassi del linguaggio, abbiamo richiamato il modulo CLA implementato in precedenza tramite la parola chiave **component** e creato quattro *istanze* di quest'ultimo dando a ognuna un'etichetta identificativa.

```
entity RC_CLA_16bit is
    Port ( A, B : in bit_vector (15 downto 0);
          S : out bit_vector (15 downto 0);
          cout : out bit);
end RC_CLA_16bit;

architecture MyRC_CLA_16bit of RC_CLA_16bit is
    signal c : bit_vector (4 downto 0);
    component CLA4bit is
        Port ( A, B : in bit_vector (3 downto 0);
              cin : in bit;
              cout : out bit;
              S : out bit_vector (3 downto 0));
    end component;
begin
    c(0) <= '0';
    CLA0 : CLA4bit port map ( A(3 downto 0), B(3 downto 0), c(0), c(1), S(3 downto 0)
    → );
    CLA1 : CLA4bit port map ( A(7 downto 4), B(7 downto 4), c(1), c(2), S(7 downto 4)
    → );
    CLA2 : CLA4bit port map ( A(11 downto 8), B(11 downto 8), c(2), c(3), S(11 downto
    → 8) );
    CLA3 : CLA4bit port map ( A(15 downto 12), B(15 downto 12), c(3), c(4), S(15
    → downto 12) );
    cout <= c(4);
end MyRC_CLA_16bit;
```

Riportiamo in basso, per chiarezza, l'immagine del sommatore richiesto dalla consegna di progetto a cui abbiamo fatto riferimento. Lo schema logico ottenuto da Vivado sarà invece analizzato nella prossima sezione.

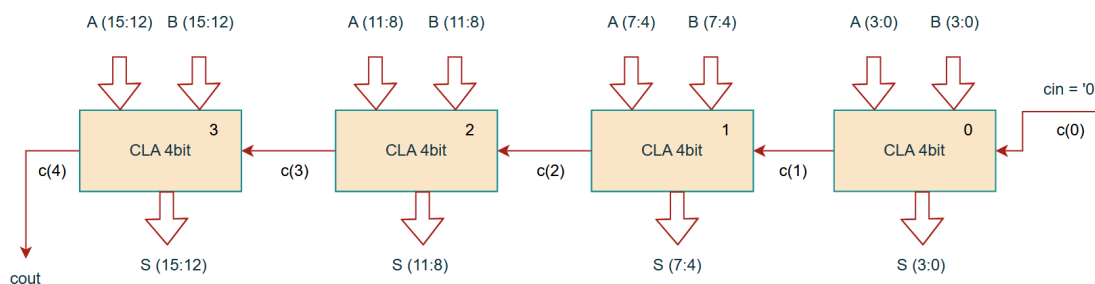


Figure 5: Sommatore richiesto

3 Schemi logici

Prima di passare alla successiva fase di simulazione logica del circuito, sfruttando la sezione “Elaborated Design” del tool Vivado, andiamo a realizzare gli schemi logici associati alle descrizioni che abbiamo realizzato mediante il linguaggio VHDL. Questa operazione ci consente di svolgere una prima veloce verifica sulla correttezza di quanto abbiamo scritto.

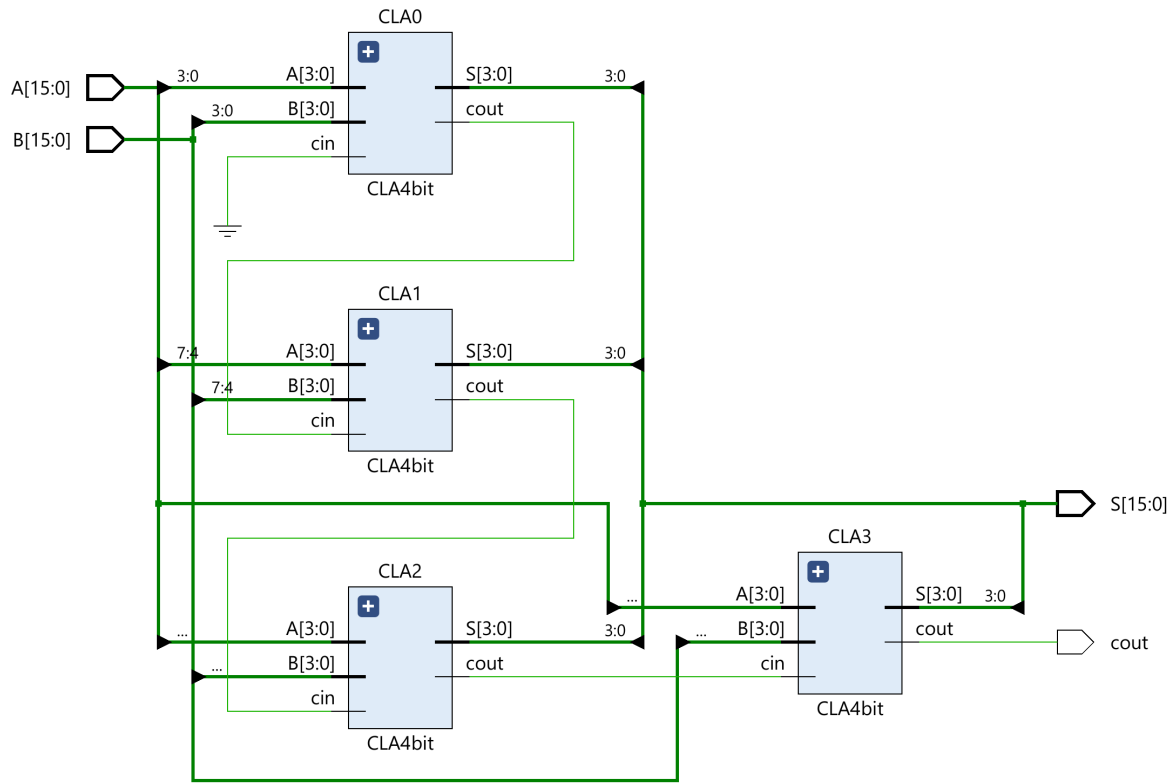


Figure 6: Schema logico raffigurante la nostra descrizione strutturale del sommatore

Una prima osservazione che riusciamo a svolgere e che, almeno a livello visivo, sembra rispettato lo schema che ci eravamo prefissati in fase implementativa. Però non siamo ancora in grado di dire nulla per quanto riguarda i singoli sommatore a 4 bit, motivo per cui chiediamo al software di tracciare lo schema di uno di essi.

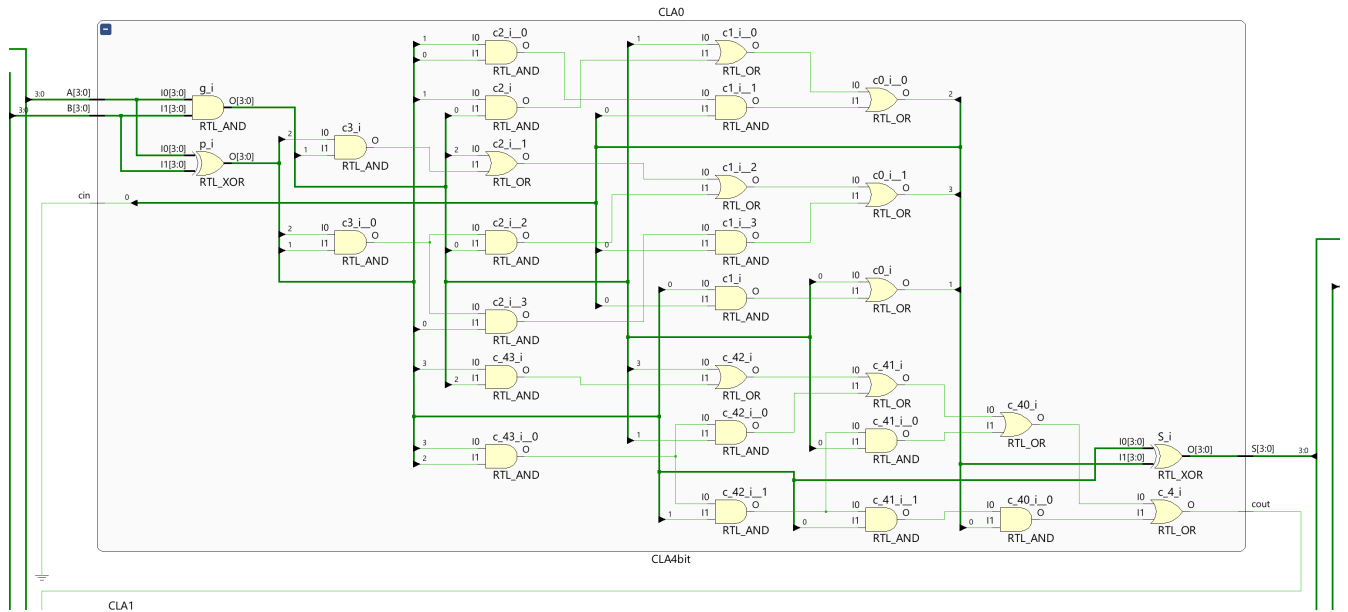


Figure 7: Schema logico del singolo Carry Look Ahead a 4 bit

In questo caso il risultato ottenuto è di più difficile interpretazione rispetto al precedente, ma in ogni caso la struttura generale sembra essere in linea con i nostri desiderata. Se chiedessimo al software di espandere gli schemi logici di ciascun sommatore a 4 bit, otterremmo un risultato equivalente a quello raggiungibile attraverso una descrizione di tipo behavioural, che, come si può notare dalla figura seguente, risulterebbe piuttosto lunga e complessa da realizzare.

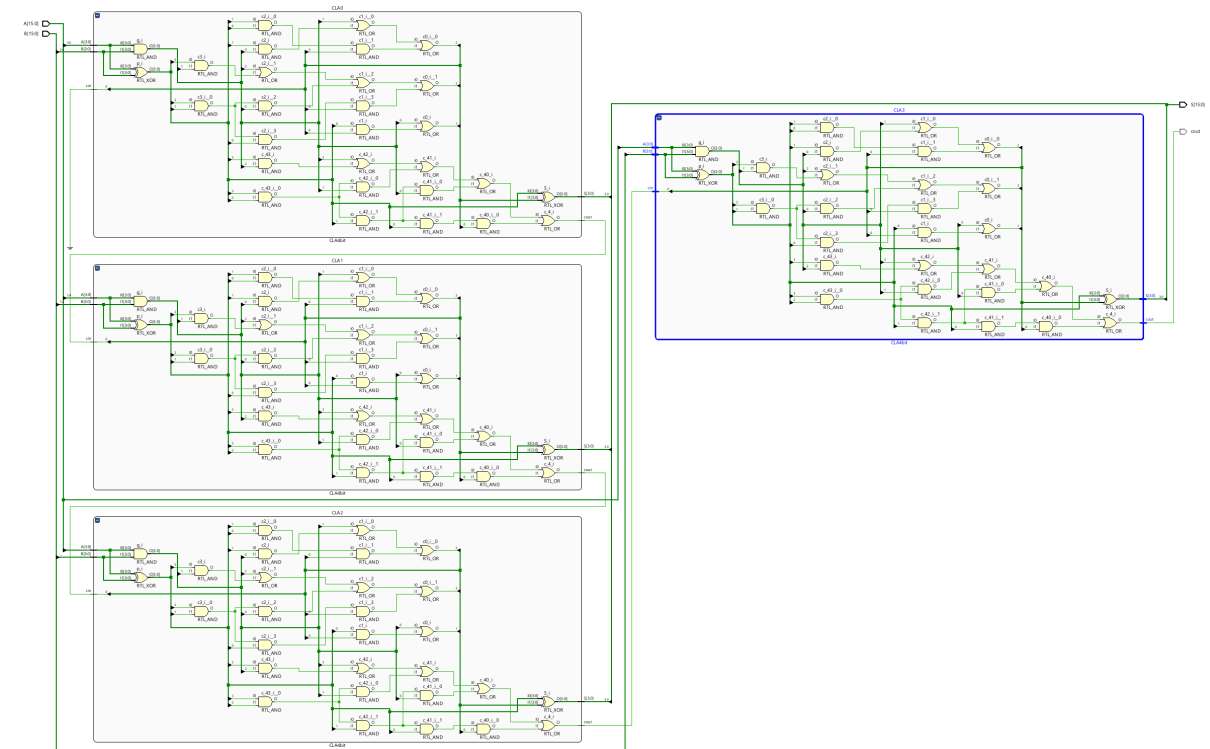


Figure 8: Schema logico analogo a quello ottenibile da una descrizione comportamentale

4 Simulazione logica

Successivamente alla verifica degli schemi logici generati dal tool Vivado, passiamo alle simulazioni del comportamento logico dei circuiti descritti, al fine di controllare che i risultati da essi prodotti seguano la matematica di un sommatore a 4 e a 16 bit (unsigned). I test svolti sono stati selezionati con attenzione al fine di rappresentare casi significativi, in modo tale da garantire la correttezza non solo delle singole somme svolte, ma anche della gestione dei riporti.

4.1 Modulo CLA a 4 bit

In prima battuta siamo andati a realizzare un test bench in VHDL per la verifica del singolo modulo CLA a 4 bit, fissando per tutte le prove il valore di cin a '0' e variando nel corso del tempo (ogni 10 ns) i valori di A e di B in modo strategico. Per poter fare quanto descritto sopra abbiamo dovuto impiegare un nuovo costrutto, il process, che, diversamente da tutto il codice VHDL fino ad adesso visto, ha natura sequenziale, ossia le istruzioni da esso contenute verranno eseguite nell'ordine in cui sono specificate.

```
entity SimCLA is
-- Port ( );
end SimCLA;

architecture MySimCLA of SimCLA is
    signal A, B, S : bit_vector(3 downto 0);
    signal cin, cout : bit;
    component CLA4bit is
        Port ( A, B : in bit_vector (3 downto 0);
              cin : in bit;
              cout : out bit;
              S : out bit_vector (3 downto 0));
    end component;
begin
    CUT : CLA4bit port map ( A, B, cin, cout, S );
    cin <= '0';
    process begin
        -- No overflow
        A <= "0000";
        B <= "0000";
        wait for 10ns;
        A <= "1111";
        B <= "0000";
        wait for 10 ns;
        -- Tutti overflow
        A <= "1111";
        B <= "1111";
        wait for 10 ns;
        A <= "1111";
        B <= "0001";
        wait for 10 ns;
        -- overflow msb
        A <= "1000";
        B <= "1000";
        wait for 10 ns;
        -- zeri ed uni alternati
        A <= "1010";
```

```

    B <= "0101";
    wait for 10 ns;
end process;
end MySimCLA;

```

Successivamente a ciò andiamo a svolgere la vera e propria simulazione impiegando Vivado, fornendogli come tempo di simulazione 60 ns

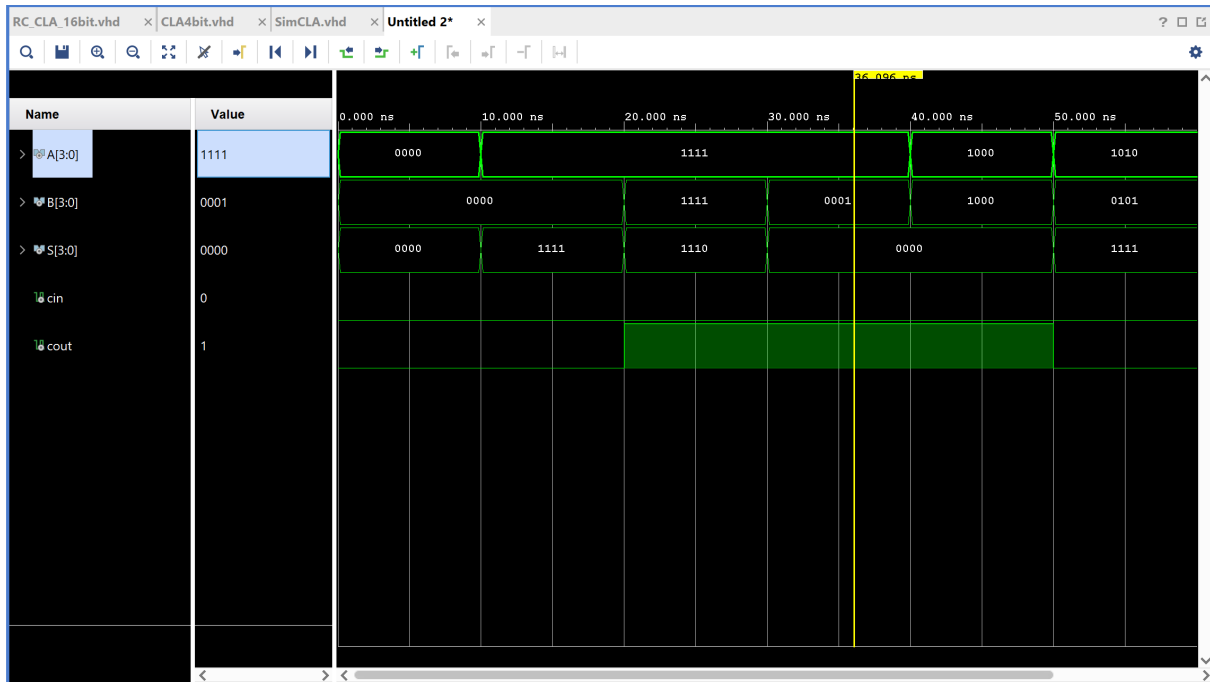


Figure 9: Esito della simulazione logica sul CLA4bit

Dagli esiti della simulazione non risultano evidenziarsi errori dal momento che sia il vettore S che il segnale c_{out} risultano essere coerenti con quanto ci si aspetterebbe dall'operazione. A titolo di esempio si può considerare il caso evidenziato con la sbarra gialla: si tratta del caso $1111 + 0001$. Ci aspettiamo come risultato 10000 , numero espresso dall'attivazione del bit di riporto c_{out} e dall'avere tutti i bit di S a zero. Questo è proprio ciò che vediamo nell'immagine.

4.2 Sommatore completo

Nota la correttezza del sommatore a 4 bit, proseguiamo nelle verifiche analizzando il sommatore a 16 bit composto da quattro CLA collegati in cascata.

Anche in questo caso sono state impiegate strategie e ragionamenti analoghi nella scrittura del test bench.

```
entity SimRC_CLA_16bit is
-- Port ( );
end SimRC_CLA_16bit;

architecture MySimRC_CLA_16bit of SimRC_CLA_16bit is
    signal A, B, S : bit_vector (15 downto 0);
    signal cout : bit;
    component RC_CLA_16bit
        Port ( A, B : in bit_vector (15 downto 0);
              S : out bit_vector (15 downto 0);
              cout : out bit);
    end component;
begin
    RC_CLA : RC_CLA_16bit port map (A, B, S, cout);
    process begin
        -- No overflow
        A <= ( others => '0' );
        B <= ( others => '0' );
        wait for 10ns;
        -- Tutti overflow
        A <= ( others => '1' );
        B <= ( others => '1' );
        wait for 10ns;
        B <= ( others => '0' );
        B(0) <= '1';
        wait for 10ns;
        -- overflow msb
        A <= "1000000000000000";
        B <= "1000000000000000";
        wait for 10 ns;
        -- zeri ed uni alternati
        A <= "1010101010101010";
        B <= "0101010101010101";
        wait for 10ns;
        -- valori casuali
        A <= "0001001000110100";
        B <= "0100001100100001";
        wait for 10 ns;
        A <= "0000000000011101";
        B <= "0000000000010010";
        wait for 10 ns;
    end process;
end MySimRC_CLA_16bit;
```

In questo caso la simulazione che andremo a compiere sarà di 70 ns.

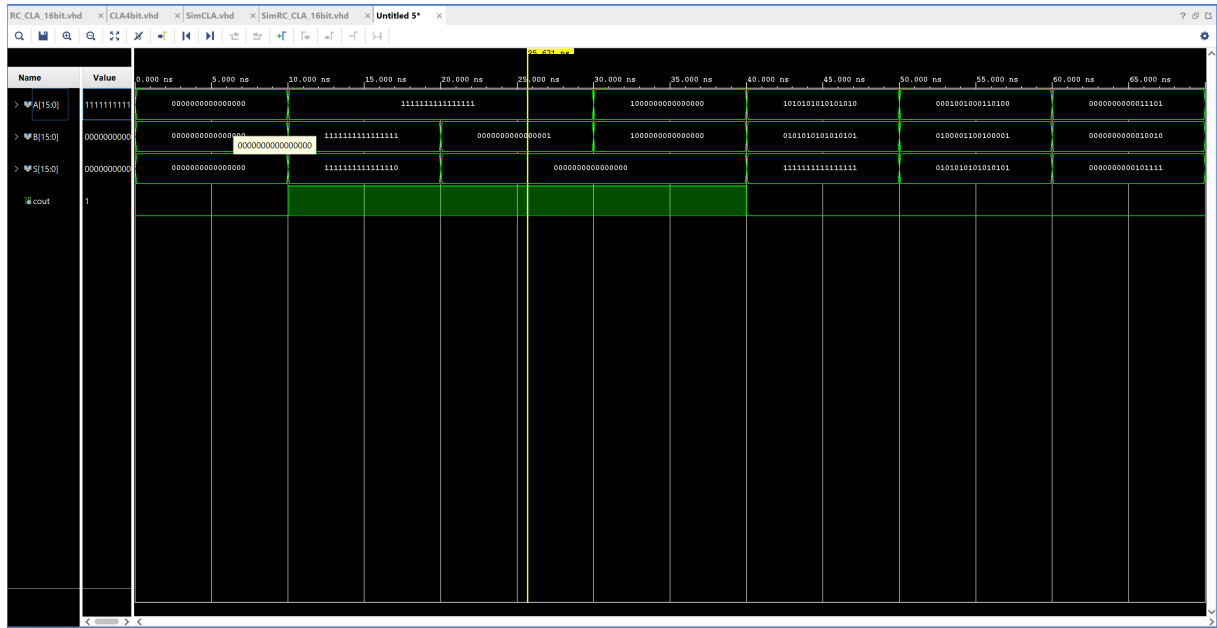


Figure 10: Esito della simulazione logica sul RC-CLA-16bit

Ancora una volta gli esiti ottenuti sono in linea con il modello matematico, in particolare la propagazione del riporto da un CLA al suo successivo.

In conclusione è abbastanza ragionevole supporre che la descrizione da noi realizzata rispetti le specifiche di progetto e che, quindi, sia idonea per la successiva fase di sintesi.